

Big Data Analysis with Hadoop and Spark: A Modern, Containerized Approach

Aymen Satouri

October 30, 2025

Contents

1	Introduction: A Creative Approach	2
1.1	The Dataset	2
1.2	The Analysis Goal	2
2	Environment: Docker vs. The Virtual Machine	2
2.1	Why I Did Not Use the Cloudera VM	2
2.2	My Solution: A Custom Docker Image	3
2.3	My Workflow: The 'Docker Commit' Strategy	3
3	Implementation: The Four Analysis Tools	3
3.1	Task 1: MapReduce (Python)	3
3.2	Task 2: Apache Hive	3
3.3	Task 3: Apache Pig	4
3.4	Task 4: Apache Spark	4
4	Final Results	4
5	Conclusion: A Portable Analysis Tool	5

1 Introduction: A Creative Approach

This project implements a full Big Data analysis pipeline using the Hadoop ecosystem (MapReduce, Hive, Pig) and Apache Spark. However, I wanted to take a more creative and modern approach.

You will find out why this project is a bit different. Instead of following the recommendation to use the older, pre-built Cloudera VM, I chose to build the environment from scratch in a portable, lightweight Docker container. This report documents the journey, the technical challenges, and the final, reusable analysis tool that was created.

1.1 The Dataset

The data source is a large, real-world dataset from Kaggle: "1 million Reddit comments from 40 subreddits."

- **File:** `kaggle_RC_2019-05.csv` (renamed to `reddit_comments.csv`)
- **Size:** ~186 MB
- **Format:** A 4-column CSV file with the headers: `subreddit`, `body`, `controversiality`, `score`.

The choice of Reddit data is interesting because it's unstructured, "dirty" text data that reflects real public sentiment, making it a perfect candidate for big data analysis.

1.2 The Analysis Goal

The goal was to filter this large dataset to find all comments that mention a specific set of keywords: `jew`, `jews`, or `jewish` (case-insensitive).

For each subreddit, we calculate the following metrics:

1. **Total Mentions:** The total count of comments containing the keywords.
2. **% Controversial:** The percentage of those comments flagged as controversial (`controversiality == 1`).
3. **% Upvoted:** The percentage with a positive score (`score > 0`).
4. **% Downvoted:** The percentage with a negative score (`score < 0`).

2 Environment: Docker vs. The Virtual Machine

The project brief recommended using the Cloudera QuickStart VM. I researched this option and quickly decided against it.

2.1 Why I Did Not Use the Cloudera VM

The Cloudera QuickStart VM is over 8 years old, no longer supported, and is based on a product (CDH 5.13) that has been end-of-life for many years. Even Cloudera's own documentation advises users not to use it. Running such an outdated and vulnerable piece of software seemed like a poor foundation for a modern data project.

I believed the core goal of this project was to learn the *tools* (MapReduce, Hive, Pig, Spark), not to fight with an obsolete virtual machine.

2.2 My Solution: A Custom Docker Image

I decided to use Docker, which is the modern standard for portable development.

1. I found a base image: `suhothayan/hadoop-spark-pig-hive:2.9.2`. This image contained all the necessary services in a lightweight environment.
2. I started the container, exposing the necessary ports (50070 for HDFS, 8088 for YARN, etc.), which allowed me to monitor HDFS and job progress from my host machine's browser.
3. I loaded my local CSV dataset into the container.

2.3 My Workflow: The 'Docker Commit' Strategy

At first, I mounted my `/code` directory and CSV file as a Docker volume. This is standard practice.

However, I was worried that if my container crashed or was accidentally deleted, I would lose all my work, scripts, and configuration. People using a VM don't have this fear, as the VM's state is persistent.

To solve this, I adopted a "commit" strategy.

1. I moved the CSV data from the mounted volume into the container's HDFS.
2. I kept all my scripts (`mapper.py`, `hive_analysis.sql`, etc.) in the `/code` directory *inside* the container.
3. After successfully completing a step (like getting the MapReduce job to work), I would commit the container's current state to a new Docker image, e.g., `my-hadoop-dev:v2-mr-done`.

This gave me "save points" and protected me from data loss. It also led to the main idea for my final project: a fully self-contained, portable analysis tool.

3 Implementation: The Four Analysis Tools

I implemented the same analysis using all four tools.

3.1 Task 1: MapReduce (Python)

I chose to write the native MapReduce job in Python instead of Java. Python is more modern, much easier to write, and less prone to boilerplate errors. The job consisted of a `mapper.py` to filter and emit metrics, and a `reducer.py` to aggregate them by subreddit.

This job worked perfectly and produced our "golden standard" result set.

3.2 Task 2: Apache Hive

Hive was challenging. A simple `LOAD DATA` with `FIELDS TERMINATED BY ','` failed, because the Reddit comment bodies (the `'body'` column) contained commas, quotes, and newlines.

The solution was to use the `OpenCSVSerde`, which is designed to handle complex, messy CSVs. This worked, but the results were **slightly different** from the native MapReduce job. Despite tweaking the query, I could not get an exact match.

I am confident this is due to a subtle difference in how Hive's `RLIKE` regex engine interprets word boundaries versus the Python `re` module. The results are very similar, but not identical.

3.3 Task 3: Apache Pig

Pig also took trial and error. Because of the complex CSV, I could not use the native `PigStorage()` loader.

The solution was to re-use my Python mapper! Pig's `STREAM` operator allows you to stream data through any external script. I streamed the raw data through my `mapper.py` script, which I had already proven works.

After that, the Pig Latin script was simple: `GROUP, FOREACH...GENERATE` to sum the values, and `STORE` to save them.

This approach worked perfectly and produced results identical to the native MapReduce job.

3.4 Task 4: Apache Spark

Spark was, by far, the easiest and most powerful tool. Using the PySpark DataFrame API, I was able to read the complex CSV file correctly in one line: `spark.read.csv(..., multiLine=True)`

The only major issue I faced was a `SyntaxError` on an "f-string" (`f"..."`). I realized this was because `spark-submit` was defaulting to the container's old **Python 2.7**.

The fix was to explicitly tell Spark to use Python 3: `PYSPARK_PYTHON=python3 spark-submit spark_analysis.py`

After that, the job ran flawlessly. **The Spark results were also identical to the native MapReduce job.**

4 Final Results

To present the work, I organized all 5 source scripts into the `/code` directory.

I also created a `/code/FINAL_RESULTS` directory. Since the container was lightweight, it did not have the `column` utility. I installed it (`apt-get update apt-get install -y bsdmainutils`) and used it to format the raw TSV output into "pretty-printed," aligned text files for easy reading.

Below are the commands to view the head of each result file.

— **1. MapReduce Results (The "Golden Standard")**

```
root@quickstart:/code# cat FINAL_RESULTS/1_mapreduce.txt | head -n 5
```

Subreddit	Total	%_Controversial	%_Upvoted	%_Downvoted
AmItheAsshole	33	9.1	90.9	0.0
Animemes	3	0.0	100.0	0.0
AskReddit	18	0.0	88.9	5.6
ChapoTrapHouse	147	0.0	87.8	10.9

— **2. Hive Results (Slightly Different)**

```
root@quickstart:/code# cat FINAL_RESULTS/2_hive.txt | head -n 5
```

Subreddit	Total	%_Controversial	%_Upvoted	%_Downvoted
news	182	9.3	79.7	15.4

todayilearned	161	6.2	80.1	10.6
ChapoTrapHouse	147	0.0	87.8	10.9
worldnews	107	12.1	78.5	12.1

— 3. Pig Results (Identical to MapReduce)

```
root@quickstart:/code# cat FINAL_RESULTS/3_pig.txt | head -n 5
```

Subreddit	Total	_%Controversial	_%Upvoted	_%Downvoted
news	182	9.3	79.7	15.4
todayilearned	161	6.2	80.1	10.6
ChapoTrapHouse	147	0.0	87.8	10.9
worldnews	107	12.1	78.5	12.1

— 4. Spark Results (Identical to MapReduce)

```
root@quickstart:/code# cat FINAL_RESULTS/4_spark.txt | head -n 5
```

Subreddit	Total	_%Controversial	_%Upvoted	_%Downvoted
news	182	9.3	79.7	15.4
todayilearned	161	6.2	80.1	10.6
ChapoTrapHouse	147	0.0	87.8	10.9
worldnews	107	12.1	78.5	12.1

(Note: The MapReduce output is not sorted by default, while the others are. The actual data is identical, as confirmed by comparing the files.)

5 Conclusion: A Portable Analysis Tool

This project was a success. The analysis was completed using four different methods, and the challenges of each were documented.

The final, "clean" Docker image (`my-hadoop-dev:v2-clean`) is the true deliverable. It contains:

- The original HDFS data (`/user/root/project/reddit_comments.csv`)
- All 5 source code scripts in `/code`
- The 4 formatted result files in `/code/FINAL_RESULTS`

This image is now a self-contained, portable "analysis appliance." I could upload this single image to Docker Hub, and anyone could download and run my analysis in seconds.

More importantly, it is now a reusable tool. Anyone can take this image, change one line in the `mapper.py` file (to replace "jew" with "trump", "biden", or "crypto" or "Satouri" (: or "Easter Egg", for example), re-run the jobs, and perform a new analysis on any Reddit dataset that follows the same format. This is the power of using a modern, containerized workflow.